AJ_24038155_dialect_v1.3_to_v1.4 update

Change specification from AJ_24038155 Dialect v1.3 to v1.4

Status

This document records the v1.3 to v1.4 update to the frozen AJ_24038155 Dialect v1.3 specification. All v1.0, v1.1, v1.2 and v1.3 lexical, syntactic and semantic rules remain normative unless explicitly changed below.

Version v1.4 is introduced because the baseline corpus contains an algorithm whose reference Python explicitly uses `isinstance(...)` to select behaviour according to runtime type. v1.3 can express the required casts and termination behaviour but cannot explicitly represent that runtime type test.

v1.4 adds runtime type inspection only. It does not add implicit coercion, general reflection, callbacks, anonymous functions, first-class function values, function references, or a general union-type system.

1. Runtime type inspection

1.1 Canonical syntax

Single target:

```
CALL ISINSTANCE(value, STRING)
```

Multiple targets:

```
CALL ISINSTANCE(value, (INTEGER, FLOAT))
```

The expression returns BOOLEAN.

1.2 Grammar extension

```
isinstance_expression ::=
    "CALL" , "ISINSTANCE" , "(" , expression , "," , type_reference , ")" ;

type_reference ::=
    runtime_type
  | "(" , runtime_type , { "," , runtime_type } , ")" ;

runtime_type ::=
    "INTEGER"
  | "FLOAT"
  | "STRING"
  | "BOOLEAN"
  | "LIST"
  | "DICTIONARY"
```

```
  | "SET"
  | "TUPLE"
  | identifier ;
```

`isinstance_expression` is added to `primary_expression`.

Type references are syntax-only. This change does not make type names ordinary expression values.

1.3 Semantics

The value expression is evaluated once. The result is true when the runtime value is an instance of one of the supplied target types and false otherwise.

For built-in types, generated Python uses Python-compatible `isinstance` semantics.

The compiler must not insert an ISINSTANCE test implicitly.

1.4 Python translation

```
CALL ISINSTANCE(value, INTEGER)
-> isinstance(value, int)

CALL ISINSTANCE(value, STRING)
-> isinstance(value, str)

CALL ISINSTANCE(value, (INTEGER, FLOAT))
-> isinstance(value, (int, float))
```

Built-in mappings:

```
INTEGER    -> int
FLOAT      -> float
STRING     -> str
BOOLEAN    -> bool
LIST       -> list
DICTIONARY -> dict
SET        -> set
TUPLE      -> tuple
```

1.5 AST and flow representation

The parser emits a dedicated `IsInstanceNode` containing the tested expression and ordered target type names. The flow representation preserves this node and renders it as a Boolean decision expression. It must not be collapsed into an unresolved ordinary function call.

2. Static semantic refinement

A directly evidenced true `ISINSTANCE` branch may conservatively narrow an untyped identifier for validation of that branch. This is required so explicitly guarded mixed-type algorithms are not rejected merely because an earlier sample assignment gives the variable a different type.

A single target may establish that target type inside the true branch. A multi-target test may preserve a compatible known type or otherwise make the value unknown for conservative validation. v1.4 does not add a general first-class union-type system.

3. Baseline Algorithm 19

The Python reference condition:

```
isinstance(var1, (int, float)) and isinstance(var2, (int, float))
```

is represented as:

```
CALL ISINSTANCE(var1, (INTEGER, FLOAT)) AND CALL ISINSTANCE(var2, (INTEGER, FLOAT))
```

The Python string branch:

```
isinstance(var1, str) and isinstance(var2, str)
```

is represented as:

```
CALL ISINSTANCE(var1, STRING) AND CALL ISINSTANCE(var2, STRING)
```

This replaces the previous baseline's unresolved `CALL IS_NUMBER(...)` / `CALL IS_STRING(...)` placeholders without changing the intended Python behaviour.

4. Reserved identifier interaction

`ISINSTANCE` is a reserved AJ keyword in v1.4. The existing source-proven restricted-identifier escape convention remains applicable when binding syntax proves that the spelling is instead an identifier.

5. Backwards compatibility

All canonical AJ v1.3 programs remain valid under v1.4.

The explicit casts `CALL INTEGER`, `CALL FLOAT`, `CALL STRING` and the `QUIT` statement retain their v1.3 meanings unchanged.

6. Version identification

This specification delta is frozen as:

Cambridge-Style Pseudocode - AJ_24038155 Dialect v1.4

Any later syntax addition, removal or semantic alteration requires a new dialect version.